

1

Software Setup & “Hello, World!”

1.1 Objectives:

The **ARM Cortex-M4** processor executes a set of 16- and 32-bit instructions, called the Thumb-2 instructions, yielding a good tradeoff between code density (16-bit instructions) and performance (32-bit instructions). In ECE 251, we’ll do both C and Assembly language programming to better understand how the architecture works. This lab will introduce the basics of the software development environment for developing and debugging programs using our ARM Cortex M4 processor and Tiva Launchpad development board. This lab will also provide some exposure to Assembly and C programming, ARM registers, and memory allocation. When you complete this lab, you should be able to:

- Understand Visual Studio Code (VSCode) and Platform.IO (a tool for embedded software developers who write software in assembly language or C for microcontrollers).
- Connect to the TM4C123G board and download a program using VSCode+Platform.IO.
- Run a program and examine changes to memory and registers.
- Use breakpoints and stepping to debug a program.

Because you will perform all these procedures in every lab after this, a complete understanding of the material in this lab is necessary.

1.2 Related Material to Read:

- NA

1.3 C Programming Fundamentals

The C language, like many other computer languages, consists of a series of statements (commands) that tell the CPU what operations to perform. Every full C program begins inside a function called “main”. A function is just a group of commands that perform some sort of task. Main can then call other functions or perform tasks using normal C programming. For example:

```
#include <stdio.h>
int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

`#include` is a “preprocessor” directive that tells the compiler to take code from `stdio.h` (part of standard C) header file and include it in our program. Preprocessor directives are commands that the compiler uses to help process some programs before compilation.

The next line `int main(void)` specifies the start of a function `main` that returns an integer (hence the `int` type). The curly braces `{}` specify the beginning and end of the function. `void` specifies that there are no inputs to the `main` function. Don’t worry if it is a bit confusing, we will go over these in much more depth.

The `printf` function is the standard C way of displaying output on the screen. This function is packaged in `stdio.h`, including that header file allowed us to call `printf`. In this case, we take the string of characters (specified by quotes “”) “Hello, world!\n” and output that to the screen. The ‘\n’ character is a special character that represents the newline character.

Lastly, we have a return statement to return a value to the part of the program that invoked this function. The `main` function is a special case where we typically return a value to the operating system (which invoked `main`) of the status of the program. A return value of 0 means success.

Now that we have some idea of what commands look like, let’s look at the tool we will be using to work with programs, which are just an ordered collection of commands, directives, and comments.

NOTE: Although `printf` is the typical way to print to a screen, our TI Tiva board is a special case. Since it runs at such a low level, we will have to use a serial protocol (UART) to achieve the same results. You’ll see how it works later in the lab.

1.4 Board Startup:

CAUTION: Your development board is sensitive to static electricity. Be sure to touch something grounded before touching the board.

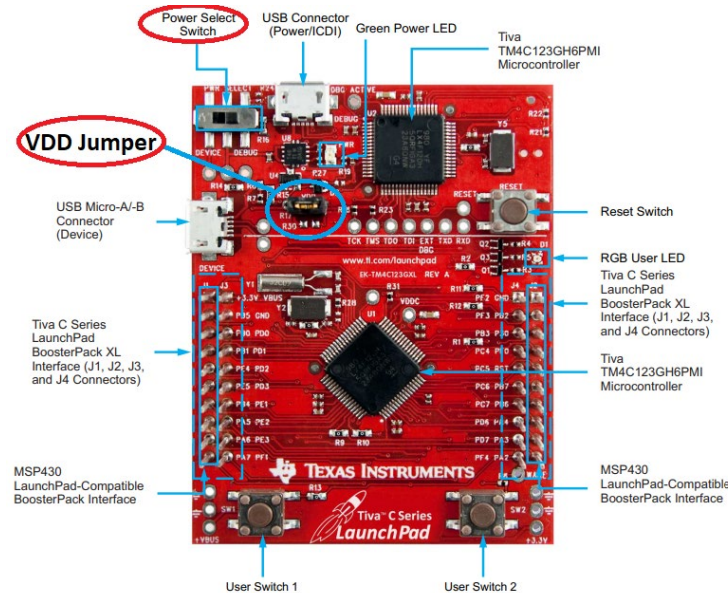
The Texas Instruments EK-TM4C123GXL is a single board computer development system for the TI TM4C123GH6PM microcontroller (one of the TM4C123 family). Support software for this development board is provided for Windows XP/Vista/7/10 operating systems. The EK-TM4C123GXL board is a small development board designed for low cost and has several common I/O ports. However, you won’t have to worry about using any I/O ports or a breadboard for several more lab sessions.

The EK-TM4C123GXL development board uses option select jumpers to configure the board operation. The jumpers should already be set for default board operation. A **“Jumper”** is a plastic encased wire shunt that connects two terminals electrically.

- **Jumper on, in, or installed:** means the jumper is installed such that the two terminal pins are connected.
- **Jumper off, out, or idle:** means the jumper is removed or installed on only one pin (i.e., there is no electrical connection between the two terminal pins through the jumper). Make sure that the jumpers are idle by installing them on *one* pin so that they will not be lost.

Now follow these steps to connect and power on the board for debugging operations:

1. Ensure the board options are in the default positions:
VDD Jumper = ON
PWR SELECT switch = DEBUG



2. Connect one end of the supplied USB cable to the development board, and the other to the USB port on your PC.

1.5 Coding in VSCode+Platform.IO

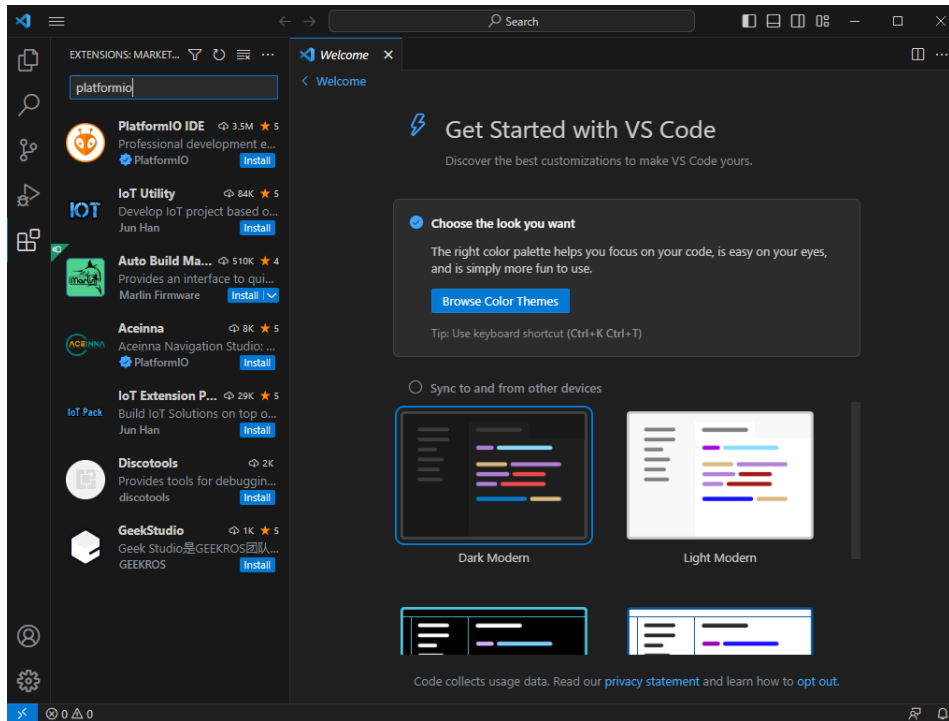
The following instructions will, to varying degrees, apply to all of the labs, so you may want to keep these instructions handy for subsequent labs. Hopefully, this will become second nature for you.

You will need to create a **lab folder** in an appropriate place in your computer file system. On your own computer a good folder path could be something like ...\\Documents\\ECE251\\Labs. Each of your ECE251 lab projects will go in *its own folder* within this folder. For this lab create a Labs folder, i.e., ...\\Documents\\ECE251\\Labs

Installing Platform.IO under VSCode:

The first time you run VSCode on a given lab system, you will most likely need to install Platform.IO. Fortunately, it’s very easy:

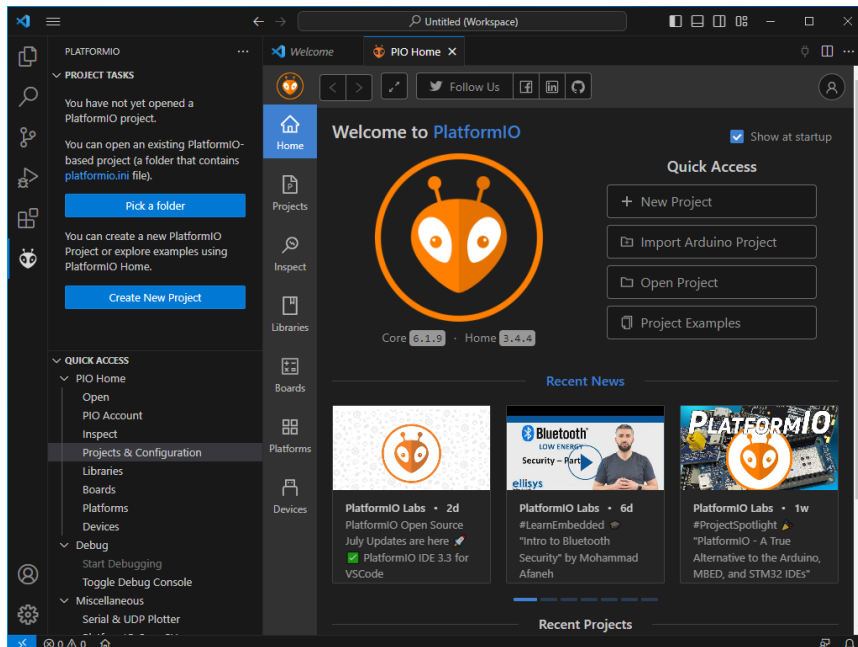
Click on the Extensions icon, , on the left side of the screen. Type in “PlatformIO” in the box:



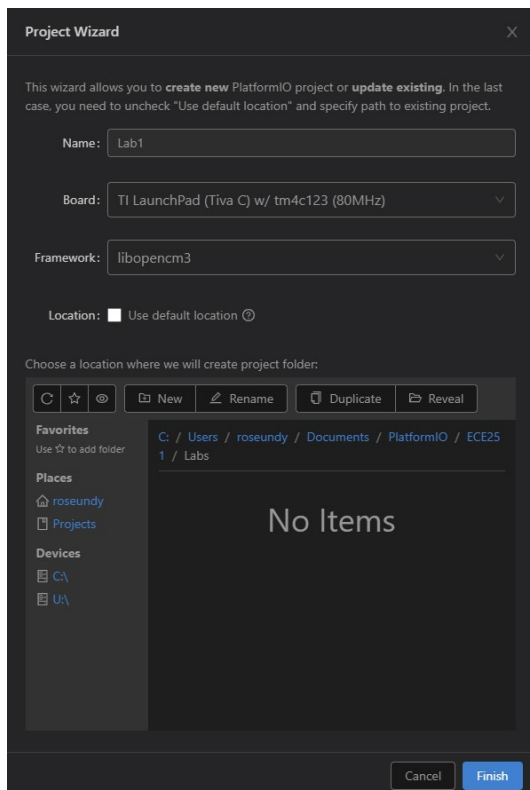
Click on “Install”. When it finishes installing, *you are done*. Our TIVA boards require a driver in order to upload and debug code, but this should already have been done on the lab systems.

Creating a Platform.IO Project:

Start VSCode, then click on the Platform.IO icon, , on the left side of the window. Click the “Create New Project” button in the left pane:

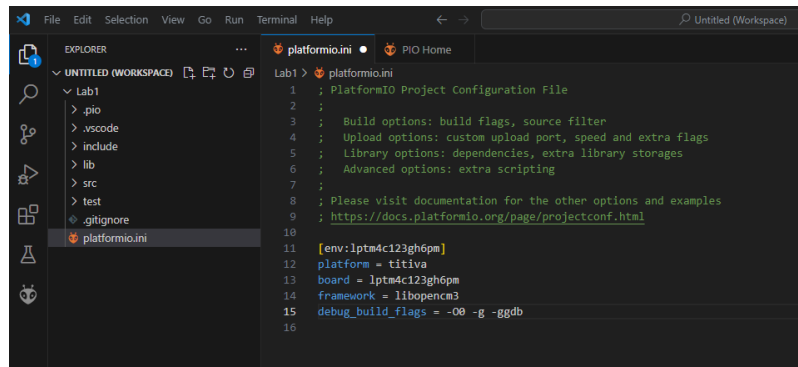


From the “PIO Home” tab, click the “+ New Project” button. This will bring up the Project Wizard. Use “Lab1” for the **Name**. Click the dropdown field next to **Board** and type in “TIVA”. Select the board that has the 80MHZ **tm4c123** listed. Select **libopencm3** for the **Framework**. Uncheck the box next to **Location**. We don’t want to use the default location. In the “Choose a location” section, navigate to the “Labs” folder you created above.



Click on the “Finish” button. The first time you create a project with the TIVA board, a number of files will be automatically downloaded by Platform.IO. This may take some time. When it finishes, VSCode will open up the Lab1 folder in the file Explorer in the left pane.

Click on the “platformio.ini” file. At the end of this file, add the line “`debug_build_flags = -O0 -g -ggdb`” (that’s the capital letter “O” followed by a zero). *Every time you create a TIVA project, you need to add this line - which makes debugging code easier.*



Adding files to the Project:

Now log into the ECE251 Canvas page, then go to Lab1 under Assignments. There you will find a file called `ECE251_Lab1_files.zip`. Download this zip file, open it, and copy the following files to the `src` folder in the `Lab1` folder that Platform.IO created under `Labs`:

```

ECE251_PIO_util.c
ECE251_PIO_util.h
main.c

```

Once you have copied the file, they will appear in the `src` folder in the VSCode file explorer.

You make click around and examine those files if you wish. Those files are the C code we’ve written for you to use in this lab. Note that one of the files is named `main.c` and contains the definition of a function called `main()`. *All projects must have a file that defines the “main()” function.*

In future labs, you can create a code file directly from the file explorer, by right-clicking on the parent folder and selecting “New file...”

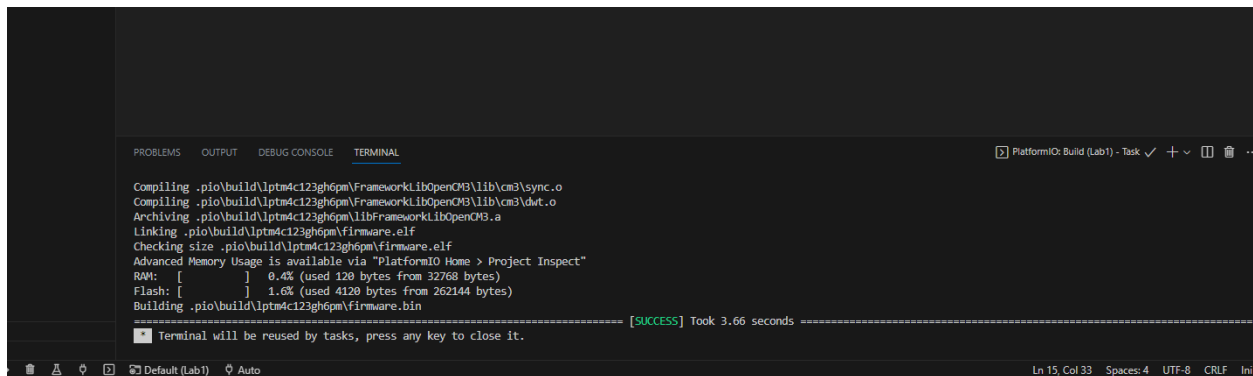
Also, you can change the behavior of Platform.IO to set a better default location for new projects (remember that box you unchecked earlier when creating your first project?). Do this by clicking on the Platform.io icon on the left side of the window, and then clicking on the “New Terminal” item in the QUICK ACCESS tree that appears. Then go to the TERMINAL tab in the bottom right pane, click on that pane, and type in “`pio settings set projects_dir U:\PlatformIO\ECE251\Labs`” (or wherever you are putting your ECE251 lab projects). You will most likely need to do this the first time you use a given system in the lab area.

Building the Project:

Once a program has been written and the platformio.ini file has been configured properly, it will need to be built. There are several of ways to do this in Platform.IO:

1. Click the Platform.IO icon, , on the left of the window. Then click on “Build” under the PROJECT TASKS tree that appears in the left pane.
2. Use the **Command Palette**. At the menu at the top of the window, use “View > Command Palette...”. Start typing “PlatformIO” in the box and it will show you a number of PlatformIO tasks. One of these is “PlatformIO: Build” (you may have to scroll down a bit). Select it.
3. Use the shortcut **Ctrl + Alt + B**
4. Click the Build button, , at the top of the currently selected code editor pane.
5. Click the Build button, , on the bottom line of the VSCode window.

If all goes well, you will see a green “SUCCESS” in the bottom right pane of the window, in the “TERMINAL” tab



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
PlatformIO: Build (Lab1) - Task ✓ + - ☐ ☐ ..

Compiling .pio\build\lptm4c123gh6pm\FirmwareLibOpenCH\lib\cm3\sync.o
Compiling .pio\build\lptm4c123gh6pm\FirmwareLibOpenCH\lib\cm3\dwt.o
Archiving .pio\build\lptm4c123gh6pm\FirmwareLibOpenCH.a
Linking .pio\build\lptm4c123gh6pm\Firmware.elf
Checking size .pio\build\lptm4c123gh6pm\Firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [ ] 0.4% (used 120 bytes from 32768 bytes)
Flash: [ ] 1.6% (used 4120 bytes from 262144 bytes)
Building .pio\build\lptm4c123gh6pm\Firmware.bin
===== [SUCCESS] Took 3.66 seconds =====
Terminal will be reused by tasks, press any key to close it.

```

If not, you will have to look at the error messages in this pane to see and correct the problem or problems in your code.

Uploading to your TIVA board:

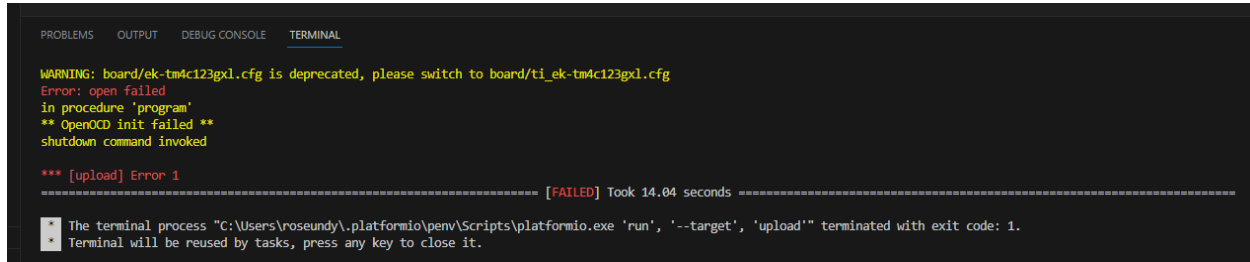
Having built your program, the next step in the process is to upload it to your board. First, connect your TIVA board to your PC with a USB micro cable. Be sure to use the USB port at the top of your board. Then, tell Platform.IO to upload the compiled code to the TIVA board. Again, there are several ways to do this:

1. Click the Platform.IO icon, , on the left side of the window, and then click “Upload” under the PROJECT TASKS tree.
2. From the command palette, as before, select “PlatformIO: Upload”.
3. Use the shortcut: **Ctrl + Alt + U**

- Click the “Run or Debug” button, , on the currently selected code editor pane and selecting Upload.
- Click the Upload button, , on the bottom line of the VSCode window.

If all goes well, you will again see the green “SUCCESS” in the bottom right pane. You can ignore the red “Error” messages in the terminal if the green SUCCESS is present. The program uploaded to the board should begin running. If not, press the reset button on the board.

If the board wasn’t connected, or there was a driver issue, you may see this:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

WARNING: board/ek-tm4c123gxl.cfg is deprecated, please switch to board/ti_ek-tm4c123gxl.cfg
Error: open failed
in procedure 'program'
** OpenOCD init failed **
shutdown command invoked

*** [upload] Error 1
===== [FAILED] Took 14.04 seconds =====

* The terminal process "C:\Users\roseundy\.platformio\penv\Scripts\platformio.exe 'run', '--target', 'upload'" terminated with exit code: 1.
* Terminal will be reused by tasks, press any key to close it.
```

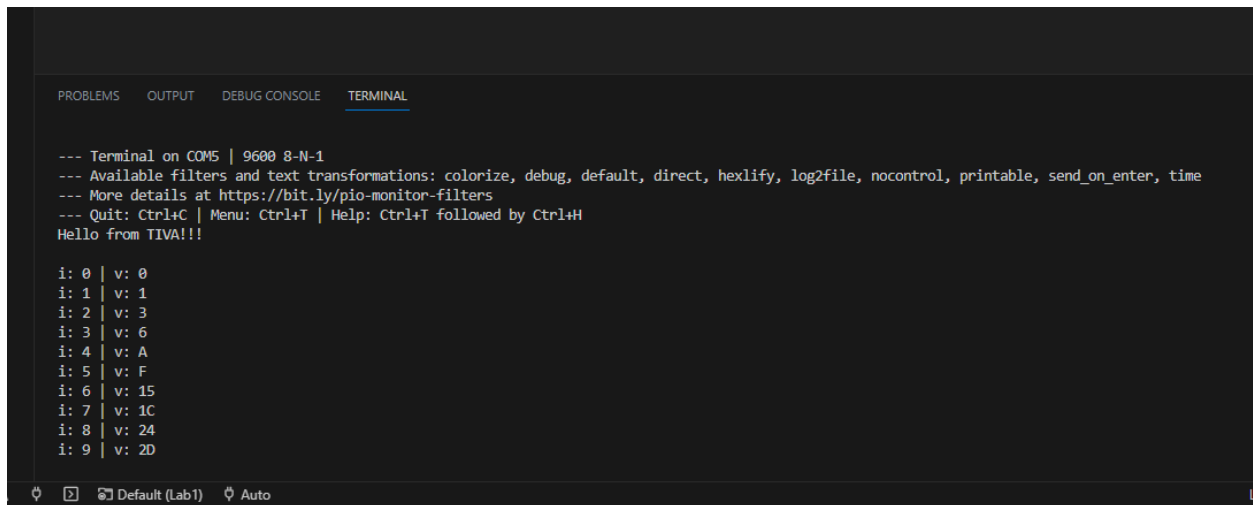
Correct the problem by connecting the TIVA board or installing the proper drivers (this shouldn’t be necessary on the systems in the lab).

Looking at the Serial Output:

To see the output of the UARTprintf function, you will need to start a Monitor window. You can get one by

- Click the Platform.IO icon, , on the left side of the window, and then click “Monitor” under the PROJECT TASKS tree.
- Select “PlatformIO: Serial Monitor” from the command palette.
- Use the shortcut: **Ctrl + Alt + S**
- Click the Serial Monitor button, , on the currently selected upper right pane or sub-pane (code editor)
- Click the Serial Monitor button, , on the bottom line of the VSCode window.

In this case, the program we are running only sends data to the serial port when it first starts up, so we need to restart the program by pressing the reset button on the TIVA board. You should then see this in the TERMINAL tab in the bottom right pane:



```
--- Terminal on COM5 | 9600 8-N-1
--- Available filters and text transformations: colorize, debug, default, direct, hexlify, log2file, nocontrol, printable, send_on_enter, time
--- More details at https://bit.ly/pio-monitor-filters
--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
Hello from TIVA!!!

i: 0 | v: 0
i: 1 | v: 1
i: 2 | v: 3
i: 3 | v: 6
i: 4 | v: A
i: 5 | v: F
i: 6 | v: 15
i: 7 | v: 1C
i: 8 | v: 24
i: 9 | v: 2D
```

Call the lab TA and demonstrate that you can build the program and load the board without reading the directions. Demonstrate that you correctly output “Hello, world!” in the Terminal tab. When you can do this without looking at the instructions, the lab TA will check you off.

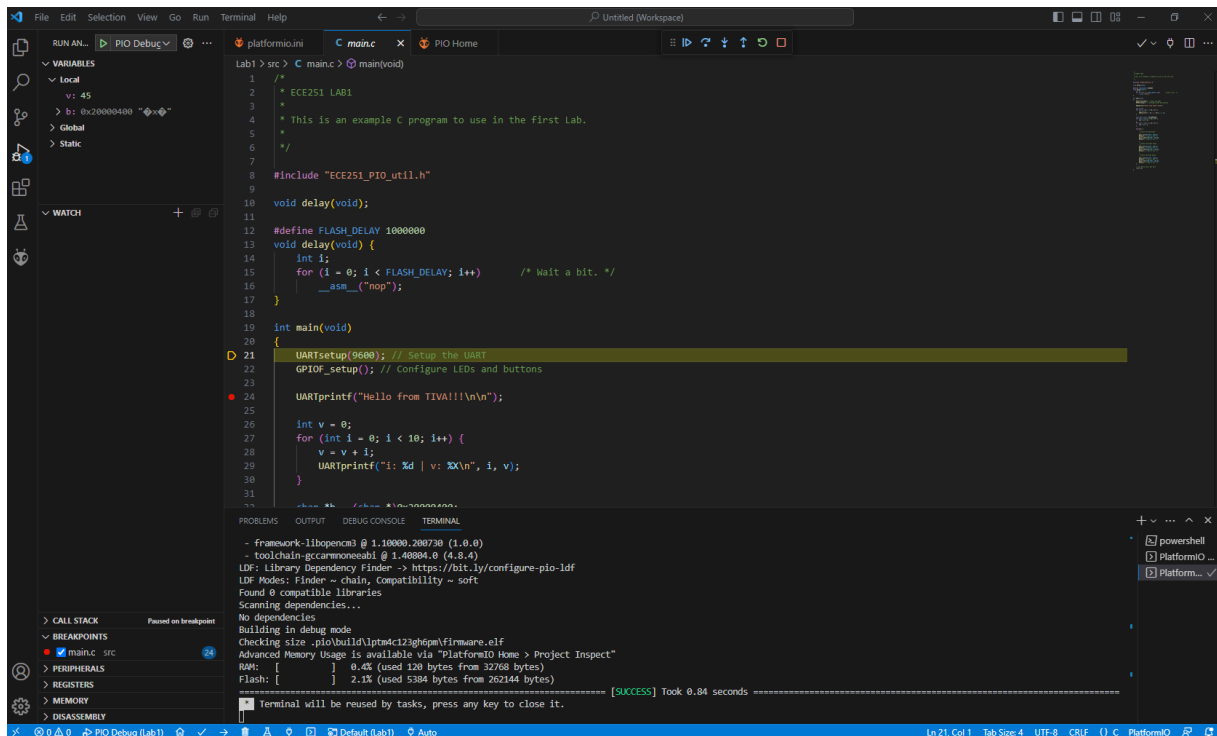
1.6 Debugging:

After a successful build/download, you can now enter debug mode in one of several ways:

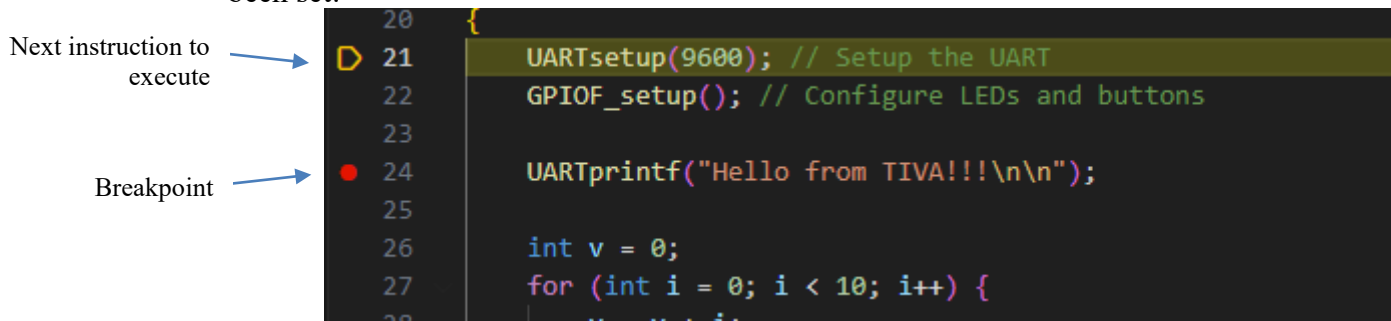
1. Click the Platform.IO icon, , on the left side of the window, and then click “Start Debugging” under the QUICK ACCESS tree.
2. Select “PlatformIO: Start Debugging” from the command palette.
3. On the top menu, select “Run”, then “Start Debugging”
4. Use the shortcut F5

This will cause your code to be recompiled with debugging options and automatically uploaded to the TIVA board. You should not start debugging until you can build and upload without errors.

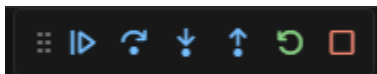
When in debugging mode, the panes being displayed in VSCode will change:



- The left pane will switch to a debugging view, giving you the ability to look at the value of C variables, the call stack, registers, memory, disassembled code, and other things.
- The bottom right pane will switch to the “DEBUG CONSOLE” showing you the status of the debugging software. For advanced users, this gives you access to *gdb* commands.
- The top right pane will switch to display the source code for the piece of code currently being executed. The position of the PC – essentially, the next instruction to be executed – is shown by a yellow arrow, . This pane will also show where any **breakpoints**, , have been set:



- The debug toolbar is shown at the top of the window:



You exit debug mode by:

1. Run > Stop Debugging
2. Pressing the red square, , in the debug toolbar
3. Use the shortcut **shift + F5**

When first entering debug mode, the program will be paused before the very first operation of your program. Typically this is the first line of code in your *main.c* or *main.s* file. The current position of the program will be indicated by the yellow arrow. Note that the current line of the program has NOT yet been executed.

Right after entering debug mode, your program does NOT automatically run. It waits at the first line of code waiting for input from you. You have the option to Run or Step through your code.

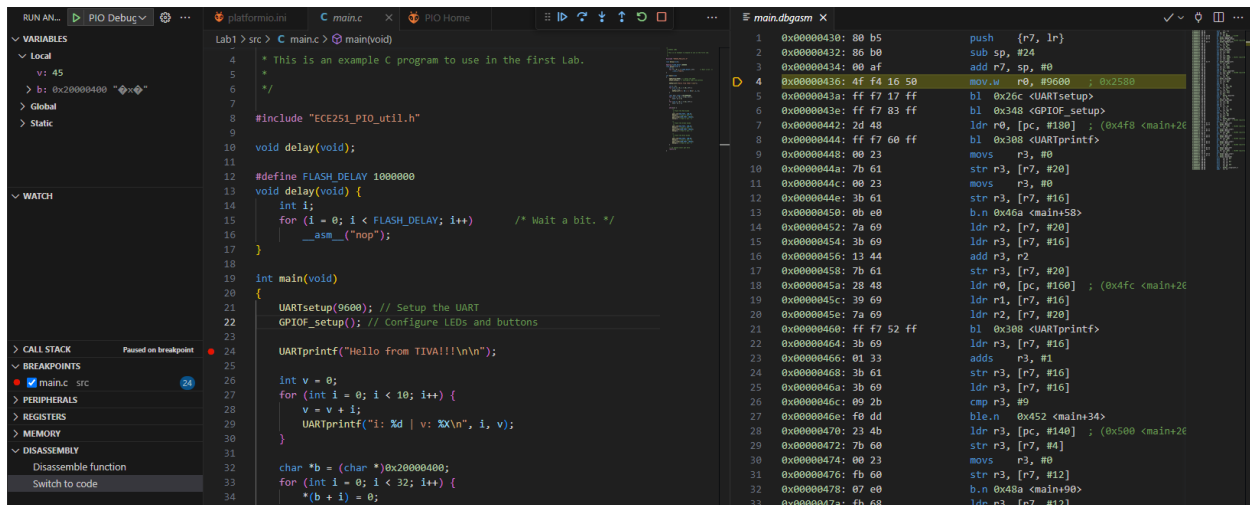
- To run the program, select Run > Continue, or the “Continue” icon, , in the debug toolbar (or F5). Selecting Run will let the program keep executing operations in real time until it either comes to a line with a breakpoint or the end of the program.
- To Step through your program line by line select Run > Step Into or the “Step Into” icon, , in the debug toolbar (or F11). Selecting Step Into will execute one line only. This will move exactly one line of code at a time (including going into functions). Functions are a sequence of instructions that performs a specific task. They are used by writing its name and providing any necessary variables (known as a function call). In *main.c*, `UARTSetup()` calls the `UARTsetup` function.
- To step through your program line-by-line but skip function calls, use the Step Over function: Run > Step Over or “Step Over” icon, , (or F10).

Breakpoints are markers you set at places you would like the program to “pause”. When a line with a breakpoint has been reached, the program stops on that line. All values in memory and registers are current allowing you to check if they are the values you expect. Note that breakpoints do not end the program. They simply “pause” it. You can continue the program by clicking Run or by Stepping.

You can add breakpoints two different ways. One way is to simply click in the area left of the line number you wish to stop on (in the same column as the yellow arrow). Another way is to place your cursor on the line you would like the program to stop on (by clicking on the desired line in the code pane), and then select Run > Toggle Breakpoint (or F9). If a breakpoint already exists, the same actions will remove the breakpoint.

Assembly vs. Code:

Instead of looking at the C source code for your program, you can instead look at the machine code (*Assembly*) for your code. Do this by clicking on “> DISASSEMBLY” on the left pane and then clicking on “Switch to assembly”. This will split the upper right pane and bring up a disassembled view of the code in that new sub-pane, or *Assembly Window*. The code will use mnemonics for the ARM Cortex-M instructions being executed on the TIVA board. The yellow arrow, showing the current instruction to be executed, will shift to this pane:



To go back to debugging via C source code, click “DISASSEMBLY > Switch to Code” on the left pane. The debugger will always use one of the two views as the active display window.

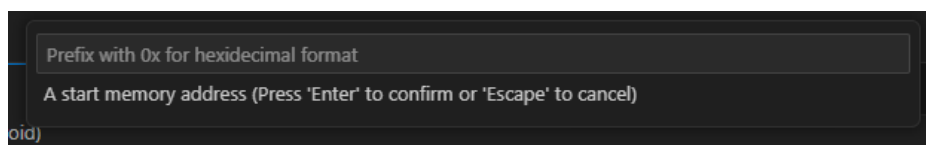
1.7 Variables, memory and registers

Variables

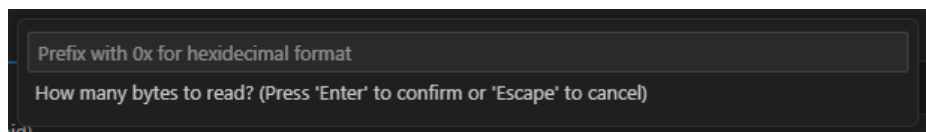
When debugging C-code (as opposed to assembly code), Platform.IO allow you to display the values of any variables used by your program. To see the values, click “> VARIABLES” on the left pane. Variables will appear under one of the “Local”, “Global” or “Static” categories.

Memory Window

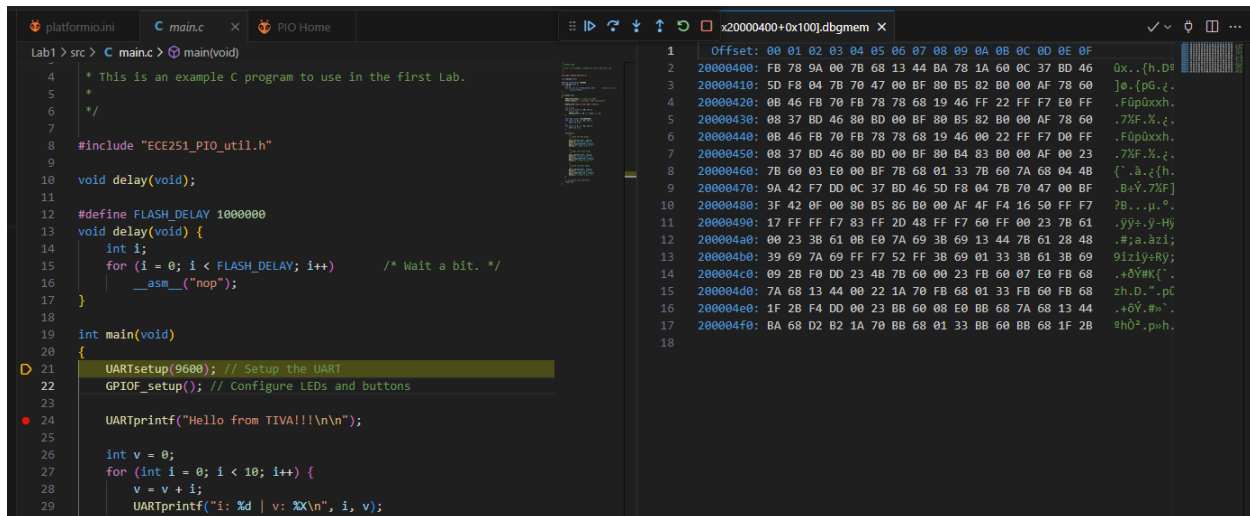
Platform.IO allows you to view the contents of any memory address using a *Memory Window*. A memory window can be opened by clicking on MEMORY in the left pane, then clicking on “Enter address...” This will bring up a small window where you can type in a memory address:



After entering the address, it will prompt you for the number of bytes to display:



For example, typing `0x20000400` and `0x100` will display 256 values in memory starting at address `0x20000400`. Note that it will add any leading zeros needed to form a 32-bit address. The memory window will be displayed by splitting the upper right pane into two parts:



The window can be closed by clicking on the “X” in the tab at the top of the pane. Note that this sub-pane may be shared by any disassembled code views. Any tab may be moved between the left and right sub-panes. In addition, any sub-pane may be further split or closed with the Split Editor button, , or “...” which appears when you select a sub-pane.

It can be seen that the memory values are grouped in twos. This is because the *Memory Window* follows the physical memory addressing design of the TM4C. Each 32-bit memory address contains an 8-bit space of memory. Each group of two hex numbers is 8-bits (1 byte), therefore each group of two hex numbers is a different address. For instance, in the example above the memory and values are as follows:

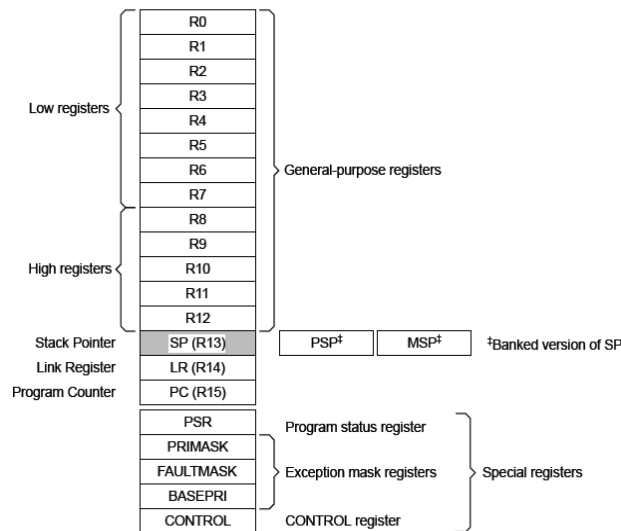
Address	Value
0x20000400	0xFB
0x20000401	0x78
0x20000402	0x9A

Currently, Platform.IO doesn’t support modifying memory from the *Memory Window*.

Registers

You can also see the current value of the processor registers by clicking on “> REGISTER” in the left pane. It functions in a similar manner to the *Memory Window*, but shows the current values of the CPU registers. Each register is 32-bits wide, so when storing register values in memory they will take up 4 address spaces in memory. Modifying registers from this window is currently not supported by Platform.IO.

Memory and Register windows are most helpful as a way to view the values of specific memory locations and registers.

Figure 1.1: The Cortex-M4F register model**Table 1.1:** Memory Map for TM4C123GXL Development Board

Address Start	Address End	Memory application
0x0000.0000	0x0003.FFFF	On-Chip Flash (<i>Where you will program</i>)
0x0004.0000	0x1FFF.FFFF	Reserved
0x2000.0000	0x2000.7FFF	Internal SRAM
0x2000.8000	0x220F.FFFF	Reserved
0x2210.0000	0x3FFF.FFFF	Reserved

1.8 Running an application using Platform.IO:

By now you should have a project all set up and ready to go. When the program is downloaded to the board, it is stored starting at address 0x0000.0000. Enter Debug mode and step through the program. In the *Memory Window*, examine memory locations 0x0000.0430 – 0x0000.0506 (**Please note that the period in the address is just to make it easier to read here. Remove the period when typing the address into the memory window**). These memory locations should contain the object code of the program. You can compare the *Assembly Window* to the values in memory. Notice in the *Assembly Window* the program is pointed to address 0x0000.0436, and the value at that address is 0x4F. Look at address 0x0000.0436 and 0x0000.0437 in the *Memory Window* and you should see 4F and F4 (the values 0x4F and 0xF4, respectively).

Now look at memory locations 0x2000.0400 – 0x2000.041F. This program modifies these memory locations by first loading 0x00 into all of them, then storing consecutive numbers starting with 0x00. Place a breakpoint at the “char *b = 0x20000400;” line (don’t forget to “Switch to code” if the yellow arrow isn’t pointed at the C source code). Run to that point (via  or F5, then step

through the program by pressing  or **F11**. Watch as the program clears the memory locations and then populates it with consecutive numbers. Watch when the code window is selected, and then switch to the assembly window, (note that stepping will only move the cursor in the C program between the “for” statement and the assignment statement, but in the *Assembly Window* will run through multiple instructions). Also pay attention to the values in the variables and the registers and how they relate to the instructions being executed.

Here we will introduce a few more debugging tools. When you start writing larger programs it won't always be practical to single-step the entire program, but you can stop at particular places by using breakpoints. After you have stepped through the program and understand how the program works, reset the program by selecting Run > Restart or the Restart icon  in the debug toolbar. This places the Program Counter back at the beginning of the program. Continue the program ( or F5) should get you back to your breakpoint. Place a breakpoint where the memory locations 0x2000.0400 – 0x2000.041F should be zeros again. Pressing  or F5 again allows the program to run to that point. Remember you can still Step after breakpoints as well.

Call the lab TA and demonstrate the changes in memory without reading the directions. When you can do this without looking at the instructions, the lab TA will check you off. Complete understanding of this lab will make future labs easier. Review any parts of this lab you are not sure about.

1.11 More Practice with Platform.IO: Loading and Running Programs

Because you will be using Platform.IO extensively throughout the course, proficiency in these tools will be key, it is highly recommended that you practice loading source programs, editing them, compiling them, building them, and debugging them. You will find another sample program accessible on the Canvas under Lab 1 called `SampleMain.c`. If you are having difficulty with the “Hello, world” program above, your TA may require that you complete the Sample Program.

Note that since both `main.c` and `SampleMain.c` define the `main()` function, the original `main.c` file will need to be moved out of the `src` folder (or, the `main()` function in that file will need to be renamed to something else).

1.12 Questions:

1. (10 pts) What are the sizes and names of the general registers on the board?
2. (15 pts) How does the reset button on the board affect the contents of RAM? How does it affect the contents of the registers? How is ROM affected?
3. (15 pts) How are single stepping and breakpoints used to debug a program? Why might you use breakpoints rather than single stepping?

1.13 Lab Report:

The lab report will always be due on the second Monday after the final week of the corresponding lab at 11:59PM. So Monday labs will be due 14 days later, Tuesday labs will be due 13 days later, and Friday labs will be due 10 days later. For the lab write up, include the following:

1. A copy of your working .c/.h files for the program. (All .c/.h files – I know you haven’t modified anything here, but let’s practice this for future labs) – 10 pts
2. A brief discussion of the objectives of the lab and the procedures performed in the lab. (PDF) – 50 pts
3. Answers to any questions in the discussion, procedure, or question sections of the lab. (PDF) – 40 pts

Submit each file to Canvas. Do not zip them together. Please keep in mind that you must have your work checked off (look for the green sections) by the TA to receive credit for the lab.